

University of Messina
Department of Mathematics and Computer Sciences,
Physical Sciences and Earth Sciences

Smart Fleet Monitor

Real-Time Fleet Tracking and Fuel Efficiency Analysis
for Rental Cars using MQTT, Python and Multi-Database Storage

Course: Database Mod B
Professor: Armando Ruggeri

Student: Fikir Mohammed Adem
Matricola: 574411

GitHub: <https://github.com/dodi-adem/smart-fleet-monitor>

Academic Year 2025/2026

1. Introduction

1.1 Project Overview

Smart Fleet Monitor is a real-time IoT data management system designed for rental car companies. The system simulates a fleet of five vehicles, each equipped with IoT sensors that continuously publish telemetry data including GPS location, speed, fuel levels and engine RPM via the MQTT protocol. This data is collected, processed and stored across three different database platforms, each chosen for its specific strengths in handling different types of data. The entire system runs inside Docker containers, making it fully portable and reproducible.

1.2 Problem Description

The growing adoption of IoT devices in the automotive sector has led to an exponential increase in data generated by connected vehicles. Rental car companies face several operational challenges that this data could help address:

Challenge	Impact
Fuel waste due to inefficient routes	Higher operational costs per vehicle
Unexpected vehicle breakdowns	Customer dissatisfaction and repair costs
No real-time fleet visibility	Inability to respond to incidents quickly
Lack of data for business decisions	Suboptimal fleet management and planning

1.3 Project Objectives

The objective of this project is to design and implement an integrated system that uses the MQTT protocol and Python to receive, process and store data from simulated IoT vehicle sensors into three different database platforms: MySQL for structured fuel and vehicle data, MongoDB for raw high-frequency telemetry, and Neo4j for graph-based route analysis across city zones. The entire system is containerized using Docker.

2. System Architecture

2.1 Data Flow

The system follows a publish-subscribe architecture centered around an MQTT broker. The data flow can be described in four stages:

Stage	Component	Description
1 - Publish	Simulator	5 cars publish GPS, fuel and engine data to Mosquitto via MQTT
2 - Broker	Mosquitto	Receives all messages and routes them to all subscribers
3 - Subscribe	Subscriber	Receives messages and routes them to the correct database
4 - Store	3 Databases	MySQL, MongoDB and Neo4j each store different data types

2.2 MQTT Topic Structure

Each vehicle publishes data on three dedicated topics. The subscriber uses wildcard subscription `fleet/#` to receive all messages and routes them based on the topic suffix:

Topic	Payload Fields
<code>fleet/{car_id}/location</code>	car_id, timestamp, lat, lon, speed_kmh, heading
<code>fleet/{car_id}/fuel</code>	car_id, timestamp, fuel_level_pct, consumption_rate
<code>fleet/{car_id}/engine</code>	car_id, timestamp, rpm, status

2.3 Docker Infrastructure

All services run inside Docker containers and communicate over a shared bridge network called `fleet_network`. This allows containers to reach each other using container names as hostnames instead of IP addresses.

```
dodi@dodi:~/Desktop/Database Mod b Project final/smart-fleet-monitor$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
fleet_mongo	mongo:6.0	"docker-entrypoint.s..."	mongo	17 seconds ago	Up 16 seconds	0.0.0
.0:27017->27017/tcp, [::]:27017->27017/tcp						
fleet_mosquitto	eclipse-mosquitto:2.0	"/docker-entrypoint.s..."	mosquitto	17 seconds ago	Up 16 seconds	0.0.0
.0:1883->1883/tcp, [::]:1883->1883/tcp						
fleet_mysql	mysql:8.0	"docker-entrypoint.s..."	mysql	17 seconds ago	Up 16 seconds	0.0.0
.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp						
fleet_neo4j	neo4j:5	"tini -g -- /startup..."	neo4j	17 seconds ago	Up 16 seconds	0.0.0
.0:7474->7474/tcp, [::]:7474->7474/tcp, 7473/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp						
fleet_simulator	smart-fleet-monitor-simulator	"python simulator.py"	simulator	17 seconds ago	Up 15 seconds	
fleet_subscriber	smart-fleet-monitor-subscriber	"python subscriber.py"	subscriber	17 seconds ago	Restarting (1)	Less than a second ago

Figure 1 — All Docker containers running

3. Technology Choices and Motivations

3.1 Eclipse Mosquitto — MQTT Broker

Eclipse Mosquitto was chosen as the MQTT broker because it is the most widely adopted open-source MQTT broker and an industry-standard choice for IoT projects. It is lightweight and runs efficiently inside a Docker container. The publish-subscribe model decouples data producers from data consumers — the simulator does not know about the databases and the subscriber does not know about the cars. This makes the system scalable since adding more vehicles requires no changes to existing components.

3.2 MySQL — Structured Data Storage

MySQL was chosen to store structured, aggregated business data. Fuel consumption records and vehicle registry data fit naturally into a relational schema with well-defined columns and foreign key relationships. MySQL's ACID compliance ensures data integrity. The schema consists of three tables: vehicles for the fleet registry, trips for per-trip summaries and fuel_logs for timestamped fuel readings.

3.3 MongoDB — Raw Telemetry Storage

MongoDB was selected for storing raw, high-frequency telemetry data. Each MQTT message is stored as a document which maps naturally to the JSON payload format used in MQTT. MongoDB's schema-free nature is ideal because the structure of telemetry data can evolve without requiring schema migrations. Two collections are used: location_telemetry and engine_telemetry.

3.4 Neo4j — Route Graph Analysis

Neo4j was chosen to model the road network and vehicle movement as a graph. Route optimization is inherently a graph problem. Each GPS location is stored as a node and each vehicle passage becomes a PASSED_THROUGH relationship with speed and timestamp properties. Each Location node also connects to a City zone node via a BELONGS_TO relationship, enabling zone-level analysis of fuel efficiency across North, Central and South Messina.

3.5 Docker — Containerization

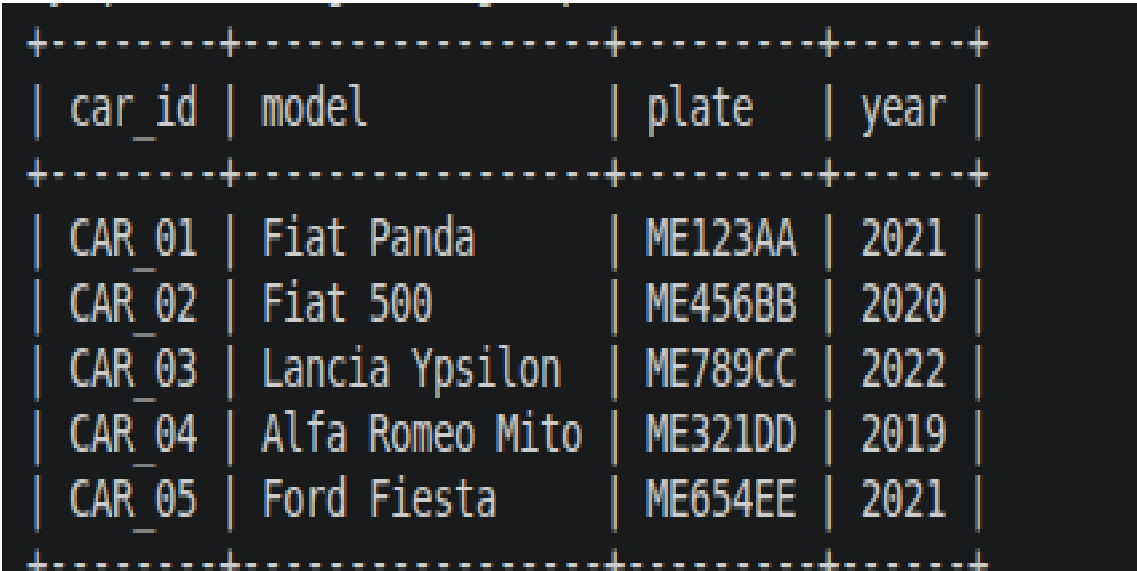
Docker was used to containerize all six components of the system. Docker Compose orchestrates all services with a single command. All services communicate over a shared Docker bridge network. Persistent volumes ensure that database data survives container restarts. The PYTHONUNBUFFERED=1 environment variable is set in the Python Dockerfiles to ensure real-time terminal output.

4. Database Design

4.1 MySQL Schema

The MySQL database `fleet_db` contains three tables. The `vehicles` table is the master registry of all cars. The `trips` table stores one row per completed trip. The `fuel_logs` table stores every fuel reading from every car, growing continuously while the simulator runs.

Table	Column	Type	Description
vehicles	car_id	VARCHAR(20) PK	Unique vehicle identifier
	model	VARCHAR(50)	Car model name
	plate	VARCHAR(20)	License plate
	year	INT	Manufacturing year
trips	trip_id	VARCHAR(50) PK	Unique trip identifier
	car_id	VARCHAR(20) FK	Reference to vehicles
	start/end_time	DATETIME	Trip duration
	distance_km	FLOAT	Total distance
	fuel_used_liters	FLOAT	Total fuel consumed
fuel_logs	efficiency_score	FLOAT	Efficiency rating
	id	INT AUTO_INCREMENT PK	Unique log entry
	car_id	VARCHAR(20) FK	Reference to vehicles
	timestamp	DATETIME	Reading timestamp
	fuel_level_pct	FLOAT	Current fuel percentage
	consumption_rate	FLOAT	Liters per 100km



```
+-----+-----+-----+
| car_id | model          | plate  | year |
+-----+-----+-----+
| CAR_01 | Fiat Panda     | ME123AA | 2021 |
| CAR_02 | Fiat 500       | ME456BB | 2020 |
| CAR_03 | Lancia Ypsilon | ME789CC | 2022 |
| CAR_04 | Alfa Romeo Mito | ME321DD | 2019 |
| CAR_05 | Ford Fiesta    | ME654EE | 2021 |
+-----+-----+-----+
```

Figure 2 — MySQL vehicles table with 5 rental cars

```
mysql: [warning] Using a password on the command line interface can be insecure.
```

id	car_id	timestamp	fuel_level_pct	consumption_rate
1	CAR_01	2026-06-27 08:59:21	75.38	6.67
2	CAR_02	2026-06-27 08:59:24	80.75	8.54
3	CAR_03	2026-06-27 08:59:27	78.67	6.36
4	CAR_04	2026-06-27 08:59:30	87.85	11.38
5	CAR_05	2026-06-27 08:59:33	99.42	11.99
6	CAR_01	2026-06-27 08:59:36	75.37	11.45
7	CAR_02	2026-06-27 08:59:39	80.72	6.09
8	CAR_03	2026-06-27 08:59:42	78.63	11.53
9	CAR_04	2026-06-27 08:59:45	87.82	10.23
10	CAR_05	2026-06-27 08:59:48	99.4	9.23

Figure 3 — MySQL fuel_logs table showing timestamped readings per car

4.2 MongoDB Collections

MongoDB stores raw telemetry in two collections inside the fleet_telemetry database. Each document corresponds to one MQTT message.

```
location_telemetry: { car_id, timestamp, lat, lon, speed_kmh, heading }
```

```
engine_telemetry: { car_id, timestamp, rpm, status }
```

```
[
  {
    _id: ObjectId('6a3f90e9ad223999c631247b'),
    car_id: 'CAR_01',
    timestamp: '2026-06-27T08:59:21.205797',
    lat: 38.115617,
    lon: 15.643978,
    speed_kmh: 60,
    heading: 237.4
  },
  {
    _id: ObjectId('6a3f90efad223999c631247d'),
    car_id: 'CAR_02',
    timestamp: '2026-06-27T08:59:24.244612',
    lat: 38.120378,
    lon: 15.650612,
    speed_kmh: 78.7,
    heading: 200.2
  },
  {
    _id: ObjectId('6a3f90f5ad223999c631247f'),
    car_id: 'CAR_03',
    timestamp: '2026-06-27T08:59:27.274502',
    lat: 38.111089,
    lon: 15.641057,
    speed_kmh: 57.5,
    heading: 66.4
  }
]
```

Figure 4 — MongoDB location_telemetry collection showing raw GPS documents

4.3 Neo4j Graph Model

Neo4j models the fleet movement as a property graph with three node types and two relationship types. GPS coordinates are rounded to 3 decimal places so nearby readings merge into the same Location node. Each Location connects to a City zone based on its latitude.

Element	Type	Properties
---------	------	------------

Car	Node	car_id
Location	Node	lat, lon (rounded to 3dp), last_seen
City	Node	name (North / Central / South Messina)
PASSED_THROUGH	Relationship	speed_kmh, timestamp
BELONGS_TO	Relationship	structural link only

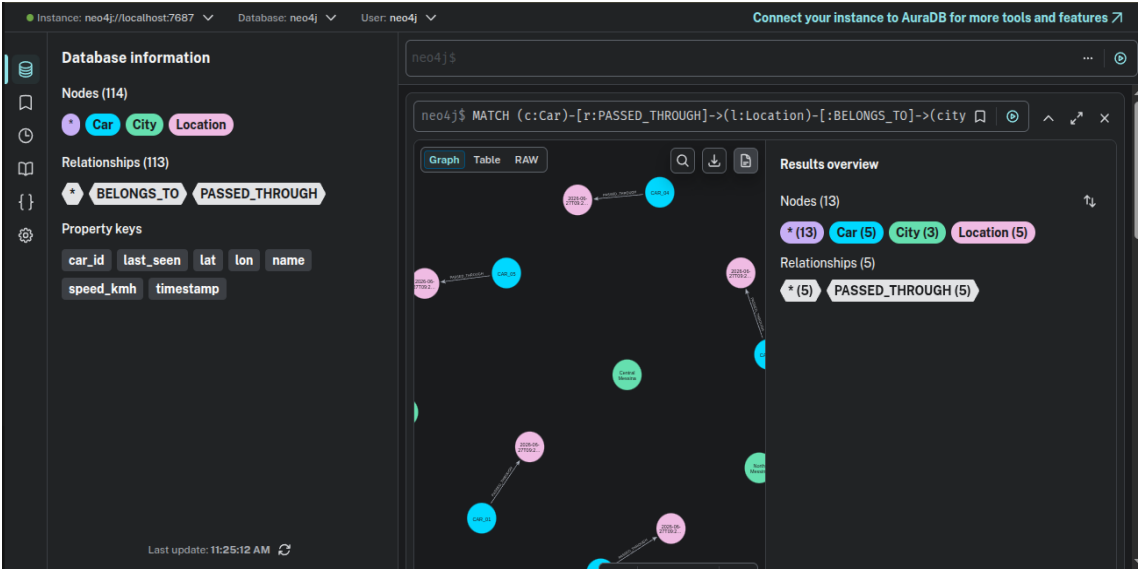


Figure 5 — Neo4j graph showing Car nodes, Location nodes and City zones

5. Implementation

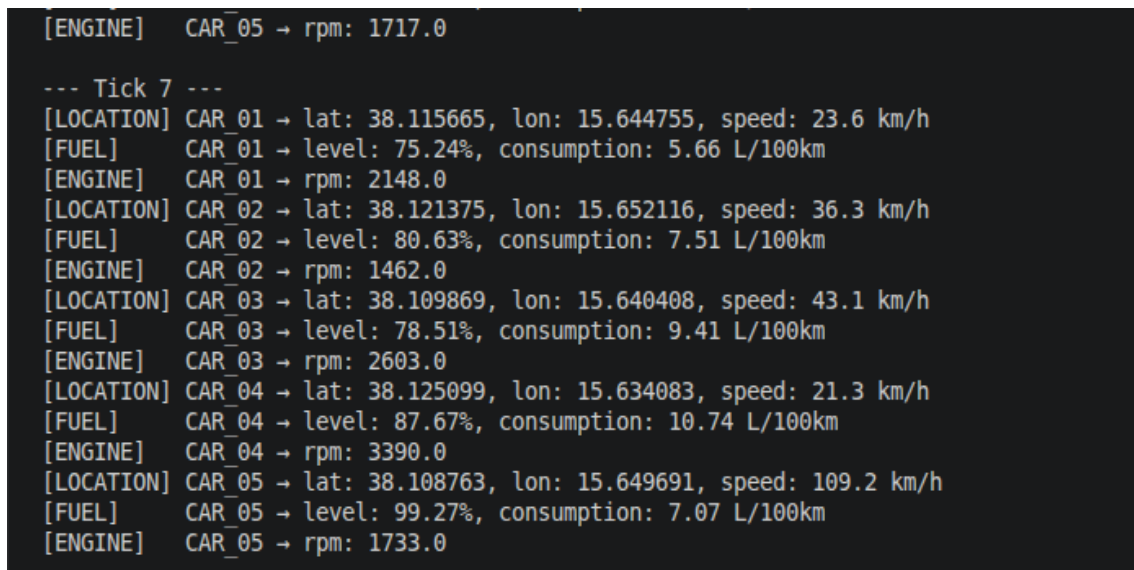
5.1 Project Structure

The project is organized into the following directory structure:

```
smart-fleet-monitor/  
  docker-compose.yml  
  mosquitto/config/mosquitto.conf  
  db/init.sql  
  simulator/  
    simulator.py | Dockerfile | requirements.txt  
  subscriber/  
    subscriber.py | Dockerfile | requirements.txt  
  handlers/  
    mongo_handler.py | mysql_handler.py | neo4j_handler.py
```

5.2 Simulator

The simulator uses a single sequential loop to publish data from all 5 cars one by one. Each car publishes location, fuel and engine data every 3 seconds before moving to the next car. A tick counter and separator line are printed between each round making the output clean and readable.



```
[ENGINE] CAR_05 → rpm: 1717.0  
  
--- Tick 7 ---  
[LOCATION] CAR_01 → lat: 38.115665, lon: 15.644755, speed: 23.6 km/h  
[FUEL] CAR_01 → level: 75.24%, consumption: 5.66 L/100km  
[ENGINE] CAR_01 → rpm: 2148.0  
[LOCATION] CAR_02 → lat: 38.121375, lon: 15.652116, speed: 36.3 km/h  
[FUEL] CAR_02 → level: 80.63%, consumption: 7.51 L/100km  
[ENGINE] CAR_02 → rpm: 1462.0  
[LOCATION] CAR_03 → lat: 38.109869, lon: 15.640408, speed: 43.1 km/h  
[FUEL] CAR_03 → level: 78.51%, consumption: 9.41 L/100km  
[ENGINE] CAR_03 → rpm: 2603.0  
[LOCATION] CAR_04 → lat: 38.125099, lon: 15.634083, speed: 21.3 km/h  
[FUEL] CAR_04 → level: 87.67%, consumption: 10.74 L/100km  
[ENGINE] CAR_04 → rpm: 3390.0  
[LOCATION] CAR_05 → lat: 38.108763, lon: 15.649691, speed: 109.2 km/h  
[FUEL] CAR_05 → level: 99.27%, consumption: 7.07 L/100km  
[ENGINE] CAR_05 → rpm: 1733.0
```

Figure 6 — Simulator terminal showing cars publishing data tick by tick

5.3 Subscriber and Message Routing

The subscriber connects to all three databases at startup. When a message arrives the `on_message` callback reads the topic and routes the payload to the appropriate handler:

Topic Pattern	Handler	Database
fleet/+/location	mongo_handler + neo4j_handler	MongoDB + Neo4j
fleet/+/fuel	mysql_handler	MySQL
fleet/+/engine	mongo_handler	MongoDB

```
o dodi@dodi:~/Desktop/Database Mod b Project final/smart-fleet-monitor$ docker logs fleet_subscriber --tail 20 -f
[MYSQL] Fuel log stored for CAR_01
[MQTT] Received message on topic: fleet/CAR_01/engine
[MONGO] Engine stored for CAR_01
[MQTT] Received message on topic: fleet/CAR_02/location
[MONGO] Location stored for CAR_02
[NEO4J] Location node stored for CAR_02
[MQTT] Received message on topic: fleet/CAR_02/fuel
[MYSQL] Fuel log stored for CAR_02
[MQTT] Received message on topic: fleet/CAR_02/engine
[MONGO] Engine stored for CAR_02
[MQTT] Received message on topic: fleet/CAR_03/location
[MONGO] Location stored for CAR_03
[NEO4J] Location node stored for CAR_03
[MQTT] Received message on topic: fleet/CAR_03/fuel
[MYSQL] Fuel log stored for CAR_03
[MQTT] Received message on topic: fleet/CAR_03/engine
[MONGO] Engine stored for CAR_03
[MQTT] Received message on topic: fleet/CAR_04/location
[MONGO] Location stored for CAR_04
[NEO4J] Location node stored for CAR_04
[MQTT] Connected to broker
[MQTT] Subscribed to fleet/#
[MQTT] Received message on topic: fleet/CAR_03/location
[MONGO] Location stored for CAR_03
[NEO4J] Location node stored for CAR_03
[MQTT] Received message on topic: fleet/CAR_03/fuel
[MYSQL] Fuel log stored for CAR_03
```

Figure 7 — Subscriber terminal showing messages being received and routed

6. Deployment

6.1 Docker Services

Container	Image	Port(s)	Role
fleet_mosquitto	eclipse-mosquitto:2.0	1883	MQTT Broker
fleet_mysql	mysql:8.0	3306	Structured data storage
fleet_mongo	mongo:6.0	27017	Raw telemetry storage
fleet_neo4j	neo4j:5	7474, 7687	Route graph storage
fleet_simulator	Custom build	-	Vehicle data publisher
fleet_subscriber	Custom build	-	MQTT consumer and DB writer

6.2 How to Run

Start the entire system:

```
docker compose up -d --build
```

View simulator output:

```
docker logs fleet_simulator --tail 0 -f
```

View subscriber output:

```
docker logs fleet_subscriber --tail 0 -f
```

Stop all services:

```
docker compose down
```

7. Conclusions

7.1 Results

The Smart Fleet Monitor successfully demonstrates the integration of MQTT messaging with three heterogeneous database platforms. The system handles data streams from five simulated vehicles, correctly routing each message type to the appropriate storage backend based on the MQTT topic. All components run in Docker making the system fully portable and reproducible with a single command.

7.2 Why Three Databases

The use of three databases is not redundancy — each serves a distinct purpose. MySQL provides ACID-compliant structured storage for business-critical fuel and vehicle data. MongoDB handles the high-frequency schema-free telemetry stream efficiently. Neo4j enables graph-native route analysis across city zones that would be extremely complex to implement in a relational or document database. Together they form a complete data platform for fleet management.

7.3 Future Developments

Several extensions could enhance the system in future iterations. A fuel efficiency scoring algorithm could rank road segments in Neo4j and suggest optimal routes to drivers. Anomaly detection on engine RPM data stored in MongoDB could trigger maintenance alerts before breakdowns occur. The simulator could be extended to read real GPS traces from public datasets. Authentication could be added to Mosquitto and all database connections for production readiness.